Reproducibility package

Few biological parameters suffice to reproduce measured dynamics: verified parameter subset selection for systems biology models

Mónica Miranda R., Carlos Salazar, César Martín, Adriana Aguirre (Escuela Superior Politécnica del Litoral, ESPOL). Submitted to *Scientific Reports*.

This package contains everything needed to check the results of the paper:

- the **Python programs** of the method and of every numerical analysis (`analysis/`);
- the **results** of those analyses, one file per benchmark problem (`results/`, 16 MB);
- the **programs that produce every figure, table and Supplementary Note** of the paper from those results (`figures_tables/`);
- the **formal proofs** in Lean 4 with Mathlib: 32 modules with 304 theorems and lemmas, and the hypotheses of 28 benchmark models (`lean_proofs/`);
- the **translator** of the SBML models into Lean and an independent check of the translation (`lean_translation/`).

The benchmark problems themselves are downloaded from the PEtab benchmark collection (`scripts/get_benchmark.sh`). The step-count data of the participant of the social cognitive theory (SCT) example are not included; they are available from the authors on reasonable request.

Three levels of checking

Level	What it does	Time	Needs
A	Regenerates every figure, table and Supplementary Note from the stored results	under a minute	Python, LaTeX
B	Compiles all formal proofs and checks that they use no unproved assumption	1–2 h	Lean (elan)
C	Re-runs the analyses from the PEtab problems and compares with the stored results	minutes per problem; about 1–2 days in total on 4 cores	Python, the benchmark (download)

Each level is independent. A reader who only wants to see where a number comes from needs level A; a reader who wants to check the mathematics needs level B; level C repeats the computations.

Contents

```
README.md               this manual
DATA_DICTIONARY.md      meaning of every field of the result files
requirements.txt        Python packages (exact versions); environment.yml for conda
LICENSE                 MIT (code); LICENSE-results.md: CC BY 4.0 (results)
CITATION.cff            how to cite
scripts/
  get_benchmark.sh      downloads the 35 PEtab problems used (commit fcbddf1, about 80 MB)
  compare_results.py    compares re-computed results with the stored ones
analysis/               the method and the numerical analyses (Python)
  config.py             paths and settings of the paper (delta, tau_R, tau_kappa, tau_e, seeds, ...)
  petab_problem.py      simulation of the measured outputs of a PEtab problem
  selection.py          parameter selection (stages 1 and 2, admissibility, backward elimination)
```

```
  external_validation.py  prediction of held-out experiments (real and synthetic data)
  robustness.py           errors at perturbation sizes 1-50 % (paired scenarios), patterns A-D
  single_parameter.py     single-parameter subsets (minimum size 2)
  simulator_check.py      comparison with the reference simulations of the benchmark
  timing.py               time of one simulation (computational cost)
  nested_validation.py    leakage-free nested validation (fixed parameters re-estimated on training data)
  synthetic_seeds.py      synthetic prediction experiment with 20 seeds (reduced and full model)
  baselines.py            random subsets and calibration-only re-estimation (baselines)
  qr_selection.py         selection by column-pivoted QR (same k, same validation)
  brun_morris.py          selection by Brun's collinearity index and by Morris screening
  profile_likelihood.py profile likelihood of every biological parameter; selection by profile
  criteria_tests.py       admissibility (+-5 %, +-20 %) and synthetic test of the subsets of all criteria
  identifiability_profiles.py    profile likelihood of the reduced problem (S and calibration free)
  identifiability_local.py       kappa and VIF of the sensitivities of S and the calibration parameters
  identifiability_conditions.py  quantities of the Lean theorem ProfileIdentifiability (I_j, gamma, remain
  identifiability_server.py/.sh  the same profiles on a many-core Linux server (Jacobian in parallel)
  crauste_blowup.py       why no local reduction exists for Crauste
  sct_model.py, sct_analysis.py   SCT example (needs the participant data)
results/                  stored results (see DATA_DICTIONARY.md)
  selection/              selected subset of each of the 34 problems
  settings/<variant>/     the same with one setting changed (15 variants, 26 problems)
  external_validation/    held-out prediction, real and synthetic data
  robustness/             errors of the 15 scenarios at 7 perturbation sizes
  single_parameter/       single-parameter subsets
  nested_validation/      leakage-free nested validation (29 problems)
  synthetic_seeds/        synthetic experiment, 20 seeds (one file per problem and seed; RESUMEN.json)
  random_subsets/, calibration_only/   baselines
  qr_selection/, brun_morris/, profile_likelihood/   selections of the other criteria
  criteria_tests/, criteria_tests_20pc/  admissibility and synthetic test of the five criteria
  identifiability/        profiles of the reduced problem (profiles/), local check (local/) and the
                          conditions of the Lean theorem (certificado*.csv); summary.md
  timing_complete/        run time of the complete method from scratch (parallel, single core,
                          and the eight first-order problems with the corrected code)
  simulator_check/, cost/, sbml_to_lean/, sct/, crauste_blowup.json
figures_tables/           produce the figures, tables and Supplementary Notes from results/
  build_all.py            runs all of them (level A)
  comparison_*.py         figures, tables and statistics of the comparison with other criteria
lean_proofs/              Lean 4 + Mathlib project (see lean_proofs/README.md)
lean_translation/
  sbml_to_lean.py         translator SBML -> Lean (writes lean_proofs/Models/)
  verify_translation.py independent check of the translated models against libRoadRunner
  mutation_control.py     control: a 1 % change of one number of a Lean file is detected
```

Installation

Python (3.11; tested on Linux with the versions in `requirements.txt`):

```
python3.11 -m venv .venv
source .venv/bin/activate            # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

or, with conda, `conda env create -f environment.yml && conda activate fewparams`.

Other tools (level A only): `pdflatex` (any TeX distribution, for Fig. 6) and `pdftops`, `pdftoppm` (poppler-utils, for the EPS and TIFF copies of the figures).

Lean (level B only): install elan; it installs the Lean version given in `lean_proofs/lean-toolchain` (4.23.0)

automatically.

All commands below are run from the folder of this README.

Level A — figures, tables and Supplementary Notes from the stored results

```
cd figures_tables
python build_all.py
```

The output is written to `figures_tables/output/`:

- `tables/table1.tex` (Table 1), `tables/tableS1.tex`, `tables/tableS3.tex`;
- `figures/fig1_pipeline.pdf` … `fig6_architecture.pdf` (with `.eps` and `.tif` copies) and the supplementary figures `figS1_validation_scatter.pdf`, `figS_robustness_scenarios.pdf`;
- `figure3/`: the consolidated table behind Fig. 3 and a summary of the counts quoted in the text;
- `suppl_*.tex`: the Supplementary Notes that contain numbers (simulator, synthetic experiment, single parameter, settings, cost, robustness, system-by-system summary, verified models, statements of the formal theorems);
- comparison with other criteria: `figures/fig3_validation_v2.pdf` (Fig. 3), `figures/fig4_comparison.pdf` (Fig. 4), the supplementary figures `figS_perfiles.pdf`, `figS_admis_mapa.pdf`, `figS_synth_semillas.pdf`, `figS_anidada.pdf`, the supplementary tables `tables/tableS_*.tex` (profile likelihood, selections, real data, Wilcoxon, admissibility, McNemar, synthetic, nested validation, run times) and, in `comparison/`, the statistics and the counts quoted in the text.

Each script prints the counts that the paper quotes (for example the numbers of admissible problems for each setting), so that they can be compared with the text. The note on the SCT example and Supplementary Fig. S3 need the participant data (place the file as described in `analysis/sct_model.py`); without it, that step is skipped with a message.

To see where a particular number of the paper comes from, use the table Where each result comes from and open the JSON file of the problem (fields explained in `DATA_DICTIONARY.md`).

Level B — the formal proofs

```
cd lean_proofs
./verify.sh
```

The script (i) checks that no file contains `sorry` or `axiom`, (ii) downloads the precompiled Mathlib, (iii) compiles the 32 theory modules and the 28 model files and (iv) prints the axioms used by every theorem. Only Lean's standard axioms (`propext`, `Classical.choice`, `Quot.sound`) must appear. The first compilation takes about 1–2 hours (mostly the largest models, Chen and Froehlich); the log of our build is `lean_proofs/BUILD_LOG.md`. `lean_proofs/README.md` relates each step of the method to its theorems, and `lean_proofs/FORMULAS.md` gives the statement of every theorem in mathematical notation.

The model files can be regenerated from the SBML models and checked independently (needs the benchmark, see level C):

```
cd lean_translation
python sbml_to_lean.py              # writes lean_proofs/Models/ (identical files) and results/sbml_to_le
python verify_translation.py        # largest relative difference with libRoadRunner: about 1e-12
python mutation_control.py          # a 1 % change of one number is detected (difference about 1e-2)
```

Level C — re-running the analyses

1. Download the benchmark problems (35 problems at commit `fcbddf1`, about 80 MB, needs `git`):

   ```
   bash scripts/get_benchmark.sh
   ```

(Another copy can be used by setting `PETAB_BENCHMARK=/path/to/Benchmark-Models-PEtab/problems`.)

2. Choose where to write the new results, so that the stored ones are kept for comparison:

```
export REPRO_RESULTS=$PWD/results_rerun      # Windows PowerShell: $env:REPRO_RESULTS="$PWD\results_r
export NPROC=4                               # processes for the problems that run in parallel
cd analysis
```

3. Run the steps in this order (each step uses the selection of step b). Any step can be limited to some problems by giving their names, e.g. `python selection.py Boehm_JProteomeRes2014`.

Step	Command	Paper	Time on 4 cores
a	`python simulator_check.py`	Supplementary Table S9	minutes
b	`python selection.py`	Fig. 2, Table 1, Suppl. Table S1	minutes, except Chen (~2 h), Lang, Raimundez
c	`for v in delta_0.001 delta_0.05 tolerances_1e-9_1e-12 tolerances_1e-6_1e-8 perturbation_0.01 perturbation_0.10 tauR_0.80 tauR_0.85 tauR_0.95 tauK_5 tauK_30 tauE_0.10 tauE_0.20 tauE_0.30 tauE_0.50; do python selection.py --variant $v; done`	Fig. 5, Suppl. Table S14	0.5–1 h per variant
d	`python external_validation.py`	Fig. 3, Suppl. Fig. S1, Suppl. Table S12	~6 h (Chen and Raimundez take most)
e	`python robustness.py`	Fig. 4, Suppl. Tables S10–S11, Fig. S2	a few hours
f	`python single_parameter.py`	Suppl. Table S13	~1 h
g	`python crauste_blowup.py`	Results (Crauste)	minutes
h	`python timing.py --write Boehm_JProteomeRes2014 Chen_MSB2009 ...`	Suppl. Table S15	minutes (times depend on the machine)
i	`python sct_analysis.py`	Suppl. Note SCT, Table S16	~30 min (needs the data)
j	`python nested_validation.py` (Chen: `VAL_PMAX=200 JAC_NPROC=110 REPRO_RTOL=1e-6 REPRO_ATOL=1e-8 python nested_validation.py Chen_MSB2009`)	Suppl. Note nested validation	hours; Chen ~10 min on 110 cores
k	`SINT_SEEDS=42-61 python synthetic_seeds.py`	Fig. 3b, Suppl. Note synthetic experiment	~1 h on 100 cores
l	`python baselines.py`	Table 2, Suppl. Note comparison on real data	~1 h

Step	Command	Paper	Time on 4 cores
m	`python qr_selection.py`	Table 2, Suppl. Note pivoted QR	minutes
n	`PL_PMAX=60 NPROC=120 python profile_likelihood.py`	Fig. 4, Table 2, Suppl. Note profile likelihood	~4 h on 120 cores (Isensee most)
o	`python brun_morris.py` (after m and n)	Table 2, Suppl. Note Brun and Morris	minutes
p	`python criteria_tests.py`, then `SOLO_SINT=1 SINT_SEMILLAS=47,...,61 python criteria_tests.py` and `ADM_NIVEL=0.20 python criteria_tests.py` (after l-o)	Fig. 4a-b, Table 2, Suppl. Notes admissibility and synthetic comparison	~1 h on 100 cores
q	`REPRO_RESULTS=$PWD/../results_timing NPROC=64 python selection.py` (and with `NPROC=1`)	Fig. 4d, Suppl. Note cost	the run time is the measurement
r	`python identifiability_local.py`, `NPROC=8 python identifiability_profiles.py` (Bachmann, Chen, Isensee, Lang, Raimundez and Smith: `bash identifiability_server.sh` on a server), then `python identifiability_conditions.py` (`CERT_PAR=256` to run the simulations of one problem in parallel)	Results (identifiability), Suppl. Note identifiability	~10 min on 8 cores; ~3 h on 256 cores for the six large problems

Steps j, n and p distribute the work over many processes and use `fork` (Linux). The stored results of steps j-q were computed on a 128-core server shared by several computations; their run times (`segundos`) are therefore upper bounds.

4. Compare each folder with the stored results:

```
cd ..
python scripts/compare_results.py results_rerun/selection results/selection
python scripts/compare_results.py results_rerun/external_validation results/external_validation
...
```

Running times and checkpoint bookkeeping are ignored; numbers are compared with a relative tolerance of 1e-6 (`--rtol` to change it).

5. Optionally, regenerate the figures and tables from the new results: `REPRO_RESULTS=$PWD/results_rerun python figures_tables/build_all.py`.

What to expect. The method is deterministic (fixed seeds), and for the small and medium problems the recomputed results agree with the stored ones to the last digits. Small differences can appear (i) in the problems whose simulations occasionally fail to integrate, so that a scenario is valid in one run and not in another (Chen: 8 to

13 of 15 scenarios per size); (ii) in the fits of the external validation, which use the result of the previous optimisation when a long fit is resumed from a checkpoint; and (iii) with other versions of libRoadRunner/SUNDIALS or of NumPy/SciPy. Running times (`segundos`, `t_sim`) depend on the machine. The comparison with other criteria (steps j-q) was computed with libRoadRunner 2.10.0; criteria that choose between nearly tied candidates can change their choice with another simulator version (for example, pivoted QR in Boehm selects `k_exp_hetero` instead of `k_phos` with libRoadRunner 2.9.2), and single local fits (one random draw of the baselines) can end in slightly different optima.

Large problems. For the eight problems in `config.FIRST_ORDER` (Alkan, Bachmann, Beer, Chen, Isensee, Lang, Lucarelli, Raimundez) the re-estimation error is computed to first order, as stated in the Methods; their sensitivity matrices are computed in parallel (`NPROC` processes), kept on disk in single precision in `results_rerun/cache/` and the computation resumes from there if interrupted. Froehlich (4,231 parameters, 9,169 experimental conditions) is not analysed on its measured outputs (each evaluation needs about 9,000 simulations); only the time of one simulation of its state trajectories is used (cost note), and its model is verified in Lean.

Where each result comes from

Paper	Program (`analysis/`)	Results (`results/`)	Figure/table program (`figures_tables/`)
Fig. 1 (method)	—	—	`make_figures.py` (`fig1`)
Fig. 2, Table 1, Suppl. Table S1; Results, numbers of selected parameters, levels of guarantee	`selection.py`	`selection/`	`make_figures.py` (`fig2`), `make_tables.py`
Fig. 3a, Suppl. Fig. S1; Results, held-out prediction	`external_validation.py`	`external_validation/`	`make_figure3.py`, `comparison_figures.py`
Fig. 3b; synthetic experiment with 20 seeds (Supplementary Note)	`synthetic_seeds.py`	`synthetic_seeds/`	`comparison_figures.py`, `comparison_tables.py`
Leakage-free nested validation (Results; Supplementary Note)	`nested_validation.py`	`nested_validation/`	`comparison_tables.py`, `comparison_suppl_figures.py`
Comparison with other criteria: Table 2, Fig. 4 and the Supplementary Notes on profile likelihood, Brun and Morris, real data, admissibility and synthetic comparison	`qr_selection.py`, `baselines.py`, `brun_morris.py`, `profile_likelihood.py`, `criteria_tests.py`	`qr_selection/`, `random_subsets/`, `calibration_only/`, `brun_morris/`, `profile_likelihood/`, `criteria_tests/`, `criteria_tests_20pc/`	`comparison_figures.py`, `comparison_tables.py`, `comparison_statistics.py`, `comparison_summary.py`, `comparison_suppl_figures.py`
Identifiability of the selected parameters (Results; Supplementary Note and Table)	`identifiability_local.py`, `identifiability_profiles.py`, `identifiability_server.py`, `identifiability_conditions.py`	`identifiability/`	`identifiability_table.py`
Run time of the complete method (Results; Fig. 4d; Supplementary Note cost)	`selection.py` run from scratch	`timing_complete/`	`comparison_tables.py`
Fig. 5, Suppl. Tables S10–S11, Suppl. Fig. S2; patterns A–D, pass rates	`robustness.py`	`robustness/`	`make_figures.py` (`fig4`), `make_robustness_note.py`
Supplementary Note on the method settings (figure and table)	`selection.py --variant`	`settings/`	`make_fig5_settings.py`, `make_extra_notes.py`

Paper	Program (`analysis/`)	Results (`results/`)	Figure/table program (`figures_tables/`)
Single-parameter subsets (Results; Suppl. Table S13)	`single_parameter.py`	`single_parameter/`	`make_extra_notes.py`
Crauste (Results, non-admissible problems)	`crauste_blowup.py`	`crauste_blowup.json`	—
Computational cost (Results; Suppl. Table S15)	`timing.py`	`cost/`	`make_extra_notes.py`
Simulator check (Methods; Suppl. Table S9)	`simulator_check.py`	`simulator_check/`	`make_extra_notes.py`
Fig. 6, Table 2; Suppl. Tables S3–S6 (formal statements)	`lean_proofs/`	—	`fig6_architecture.tex`, `make_tableS3.py`, `make_formulas.py`
Verified models, translation fidelity (Methods; Suppl. Tables S7–S8)	`lean_translation/`	`sbml_to_lean/`	`make_models.py`
System-by-system summary (Supplementary Note)	all of the above	all of the above	`make_systems_note.py`
SCT example (Introduction, Results; Suppl. Table S16, Fig. S3)	`sct_analysis.py`	`sct/`	`make_sct_note.py`

Supplementary Tables S2 (notation) and S4 (scope of each guarantee) are text of the Supplementary Information and are not generated.

Notes

- **Settings of the method.** All thresholds and seeds are in `analysis/config.py` and are those of the Methods: delta = 1 %, eta = 10 %, tau_R = 0.89, tau_kappa = tau_VIF = 10, tau_e = 0.436 (c = 0.9), k_min = 2, 15 scenarios at +-5 % with seed 42.
- **Language of the result files.** Field names in `results/` are those written by the original programs (Spanish); `DATA_DICTIONARY.md` translates them. The programs of this package write the same fields, so new and stored results can be compared directly.
- **Simulator.** libRoadRunner (CVODE, relative and absolute tolerances 1e-7 and 1e-10). Measurements at steady state and pre-equilibration use t = 1e5. If the integrator fails, the simulation is repeated piecewise with the discontinuities of the condition as breakpoints (Isensee).
- **Smith.** Its simulation time in `cost/` was measured on a second machine and scaled by 1.55 (field `nota`); all other times were measured on the same four-core server.
- **SCT data.** `analysis/sct_model.py` describes the expected file (columns `xi3`–`xi8`, `eta4`, one row per day).

Changes in version 1.2

- **Identifiability of the selected parameters.** New analyses `identifiability_*.py`, results `results/identifiability/` Lean module `lean_proofs/ProfileIdentifiability.lean` (13 theorems) and two theorems in `Poda.lean` (inclusion-minimality for a monotone criterion); the formalisation has 32 modules and 304 declarations.
- **One run for all real-data numbers.** `figures_tables/common.load_validation()` takes the test chi2/n of the reduced model, the refitted full model and theta0 from `results/nested_validation` (falling back to `external_validation` where that run has no value), for Table 1, Fig. 3 and the system summaries, which previously used `external_validation`.

- **Admissibility comparison.** Bertozzi is excluded (`common.EXCLUDE_ADMISSIBILITY`): our method selected its parameters on all measured outputs, which include the test conditions, whereas the other criteria selected on the training outputs.
- `figure_qr_scatter.py` regenerates the pivoted-QR scatter plot (Supplementary Figure).

Corrections in version 1.1

- **First-order re-estimation (`analysis/selection.py`).** With biological-only selection, version 1.0 computed the columns of the sensitivity matrix only for the biological parameters, so the first-order re-estimation used for the eight problems of `config.FIRST_ORDER` did not re-estimate the calibration parameters. The stored results were not affected (they had been computed with a cached matrix that contained those columns), but a run from scratch gave different subsets for Bachmann, Chen, Isensee, Lang and Raimundez. The columns of the calibration parameters are now computed as well; run from scratch, the program reproduces the stored subsets and `eajuste_med` of all eight problems (`results/timing_complete/parallel_corrected_code/`).
- **Attribution.** The pivoted-QR control was attributed to Brun et al. in version 1.0; it is now `qr_selection.py`, and Brun's collinearity index is implemented in `brun_morris.py`.

License and citation

Code: MIT (`LICENSE`). Results: CC BY 4.0 (`LICENSE-results.md`). Please cite the paper and this package (`CITATION.cff`). Contact: Mónica Miranda R., mmmirand@espol.edu.ec.